Merabetta UI Integration Guide

Product listing by category, subcategory and individual product ID. Verified UAT contract: 2 November 2026. Audience: CMS and storefront UI teams.

UI integration decisions

For CMS product management use /api/admin/products with category_id and an authenticated admin cookie. For public storefront browsing use /api/app/products with category and no admin login. Their IDs, filtering and pagination are different; do not interchange the contracts.

Daily Living root category is 2. Adult Diaper child category is 1. There is also a nested category named Daily Living (3), so identify the root by ID and parentId rather than name alone.

Admin filtering is direct assignments only. Public filtering includes descendants. A CMS selector that should aggregate the whole tree must fetch descendants, query each category with pagination and deduplicate product_id values. Do not add an unsupported include_descendants query parameter.

Public listings currently return no Daily Living entries. Display a normal empty state; do not substitute admin products or invent public IDs. Admin directly assigned Daily Living products currently include 583683 and 998974.

Browser request examples

Use the existing same-origin /api proxy. For the CMS, send credentials: 'include'. The browser manages the HttpOnly cookie; do not store it in localStorage or manually set a Cookie header. Login and reads must use the same browser origin/proxy configuration.

```
async function getJson(path: string) {
  const response = await fetch(path, {
    credentials: 'include',
    cache: 'no-store',
  });
  const body = await response.json();
  if (!response.ok) {
    throw new Error(body.error || `HTTP ${response.status}`);
  }
  return body;
}

// CMS: direct Daily Living assignments
const page = await getJson(
  '/api/admin/products?category_id=2&limit=100&offset=0'
);
const products = page.data;
const hasMore = page.pagination.hasMore;

// CMS: immediate children of Daily Living
const children = await getJson(
  '/api/admin/categories?parent_category_id=2&limit=100&offset=0'
);

// CMS: exact backend product ID
const product = await getJson('/api/admin/products/998974');

// Public storefront: Daily Living including descendants
const publicPage = await getJson(
  '/api/app/products?category=2&limit=100&offset=0'
);
const publicTotal = publicPage.total;
```

Preserve returned IDs as strings. Admin product_id is the backend product ID. Public id is shop:<vendor-offer-id>; public productId and variantId are distinct fields. Use the exact returned public id in a public detail URL. An individual product is wrapped in data; a list has data as an array.

Loading, pagination and errors

Show loading, empty and error states separately. A 200 with data: [] is valid. For admin lists follow pagination.hasMore; for public lists compare accumulated offset with top-level total. Advance offset by actual page length and stop/report an inconsistent empty or repeated page rather than looping.

401 means the admin session is missing or expired: send the user through the existing login flow. 404 can indicate a missing or archived product; inspect the returned error. 422 usually indicates invalid IDs or unsupported query parameters. Preserve the method, path, status and backend error in diagnostic output without logging cookies or passwords.

For changing categories rapidly, cancel obsolete requests or ignore stale responses so an earlier category response cannot overwrite the latest selection. Changing category/search should reset pagination to offset 0.

Terminal authentication troubleshooting

A cookie obtained from http://16.4.61.3:3000 belongs to 16.4.61.3 and does not authenticate api.merabetta.com. With your existing direct-server login cookie jar, these exact values work together:

```
API='http://16.4.61.3:3000'
COOKIE_JAR='/tmp/api3-server-cookies.txt'
curl -sS -b "$COOKIE_JAR" "$API/api/admin/auth/me"
curl -sS -b "$COOKIE_JAR" \
  "$API/api/admin/products?category_id=2&limit=100&offset=0"
```

For domain-based requests use https://api.merabetta.com and a cookie jar from a login to that domain. For browser integration use the CMS's existing HTTPS proxy. No passwords, cookies or access tokens are included in this handoff.

Interactive zsh may reject pasted lines beginning with #. Paste only the commands, or enable interactive comments with setopt interactivecomments. The # error does not itself cause HTTP authentication failure.

Verified curl reference

Verified against UAT at https://api.merabetta.com on 2 November 2026. All verification requests were read-only. No records or application files on UAT were changed for this documentation.

Category IDs

Use category IDs rather than matching names. The Daily Living root has ID 2, slug daily-living, and parentId: null. A separate nested category is also named Daily Living (ID 3); it is not the root.

Examples of direct children of root 2:

ID	Name
1	Adult Diaper
40	Multi-layered absorbent
98	Adult Diaper Pants
99	Adult Diaper Tape
100	Adult Incontinence Pads
116	Shower Chair

IDs and counts are current observations; retrieve categories again when building selectors.

Setup

Commands use curl; JSON filtering examples additionally require jq.

```
API='https://api.merabetta.com'
# For admin commands: path to a cookie jar from your existing admin login.
COOKIE_JAR='/path/to/admin-cookies.txt'
```

Admin endpoints require the admin_auth cookie. A missing or expired session returns 401. Keep your real cookie jar private and outside the repository. Public /api/app/products requests need no login. GET commands here need no JSON body or Content-Type header.

For CMS proxy checks, change API to https://cms.merabetta.com and use that hostname's authenticated cookie jar for admin requests. Curl honors cookie hostname/path rules; a cookie jar for the API hostname does not automatically authenticate the CMS hostname.

Admin/CMS product management

List Daily Living products, directly assigned to root category 2

```
curl --fail-with-body -sS -b "$COOKIE_JAR" \
  "$API/api/admin/products?category_id=2&limit=100&offset=0&sort_by=product_id&sort_order=asc"
```

Verified 200: total 2 at verification time, including product IDs 583683 and 998974. This does not aggregate descendant category assignments. Products assigned only to a child require querying that child.

List Daily Living's immediate subcategories

```
curl --fail-with-body -sS -b "$COOKIE_JAR" \
  "$API/api/admin/categories?parent_category_id=2&limit=100&offset=0"
```

Verified 200: total 30 immediate children. To descend another level, use the chosen child's ID as parent_category_id. Follow pagination for every level; immediate children are not the entire recursive tree.

List products assigned to a subcategory

Subcategories use exactly the same product filter. Example: Adult Diaper, ID 1:

```
curl --fail-with-body -sS -b "$COOKIE_JAR" \
  "$API/api/admin/products?category_id=1&limit=100&offset=0"
```

Verified 200: total 5. Adult Diaper Pants uses category_id=98.

There is no separate subcategory_id parameter. To aggregate the entire Daily Living branch in an admin client, retrieve every descendant category, paginate the products for each category including root 2, and deduplicate by product_id. There is no verified admin include_descendants switch.

Get one product by its backend product ID

```
curl --fail-with-body -sS -b "$COOKIE_JAR" \
  "$API/api/admin/products/998974"
```

Verified 200 for JOYLIFE DISPOSABLE ADULT DIAPERS, one of the products directly assigned to Daily Living. Product IDs are returned as strings; preserve them exactly. An archived/deleted or nonexistent product can return 404.

To inspect that product's category assignments:

```
curl --fail-with-body -sS -b "$COOKIE_JAR" \
  "$API/api/admin/products/998974" | jq '.data | {product_id, product_name, categories}'
```

There is no separate category-scoped individual product URL. Fetch the product and inspect its categories, or find its exact product_id in the fully paginated category list. Use the detail response to inspect all assignments; a category-filtered list may only populate the matching category association.

Pagination and optional search

Admin product/category list response:

```
{
  "data": [],
  "pagination": { "total": 0, "limit": 100, "offset": 0, "hasMore": false }
}
```

limit is 1–100, default 20; offset starts at 0. Continue while hasMore is true, advancing offset by the actual number of rows returned. Check for an unexpectedly empty or repeated page.

Product sorting supports product_id, product_name, product_code, sku, and sort_order=asc|desc. Text search is not an exact product-ID filter:

```
curl --fail-with-body -sS -b "$COOKIE_JAR" --get \
  "$API/api/admin/products" \
  --data-urlencode 'category_id=2' \
  --data-urlencode 'search=JOYLIFE' \
  --data-urlencode 'limit=100' \
  --data-urlencode 'offset=0'
```

Use the detail URL for exact ID lookup, not ?product_id=998974; product_id is not an accepted list filter.

Public app/storefront

List categories and select Daily Living children

```
curl --fail-with-body -sS "$API/api/app/products/categories"
```

Response is { "data": [...] }, without pagination. Fields include string id, name, slug and parentId.

```
curl --fail-with-body -sS "$API/api/app/products/categories" \
  | jq '.data[] | select(.id == "2" or .parentId == "2") | {id, name, parentId}'
```

List public products for Daily Living, including descendants

```
curl --fail-with-body -sS \
  "$API/api/app/products?category=2&limit=100&offset=0"
```

The deployed public catalog expands each product's categoryIds with ancestor IDs. Therefore root category=2 includes qualifying descendant products. Unlike admin, its parameter is category, not category_id. Pass the ID, not the category name or slug.

List public products for a subcategory

```
curl --fail-with-body -sS \
  "$API/api/app/products?category=1&limit=100&offset=0"

curl --fail-with-body -sS \
  "$API/api/app/products?category=98&limit=100&offset=0"
```

Verified root 2, child 1 and child 98 returned HTTP 200 with { "data": [], "total": 0 } at verification time. Admin records existing does not guarantee a matching public catalog entry. An empty 200 response is not a route error.

Public pagination uses top-level total, not pagination.hasMore. limit is 1–500, default 100; offset starts at 0. Advance by the number of received entries until offset reaches total. Optional filters are brand (ID) and q (text).

Get an individual public product

Use the exact id returned by the public list. On the currently deployed API its format is shop:<offer-id>. productId and variantId in the public object are separate backend identifiers; the numeric part of shop:... is a vendor offer ID, not the backend product ID.

This example first finds a public Daily Living entry and then requests its exact detail URL. It reports the current empty catalog instead of inventing an ID:

```
PUBLIC_ID=$(curl --fail-with-body -sS \
  "$API/api/app/products?category=2&limit=1&offset=0" | jq -r '.data[0].id // empty')

if [ -n "$PUBLIC_ID" ]; then
  curl --fail-with-body -sS "$API/api/app/products/$PUBLIC_ID"
else
  echo 'No public Daily Living products are currently returned.'
fi
```

Detail response is { "data": { ... } }. The deployed public detail validator currently accepts shop:<digits> only. Do not pass 998974 or product:998974 to the public detail endpoint: those are not supported by this deployed version. Use /api/admin/products/998974 with admin authentication to read that backend product directly.

A public product can also be located by its backend productId by paginating the public list and comparing its productId; multiple public offers may reference the same backend product. There is no deployed public product_id query filter.

Endpoint comparison

Purpose	Admin (cookie required)	Public (no login)
Root category products	/api/admin/products?category_id=2 (direct assignments)	/api/app/products?category=2 (includes descendants)
Child category products	/api/admin/products?category_id=1	/api/app/products?category=1
Category discovery	/api/admin/categories?parent_category_id=2 (paginated immediate children)	/api/app/products/categories (all active public categories)
Exact product	/api/admin/products/998974	/api/app/products/<returned-shop:id>
List pagination	data + pagination	data + top-level total

All commands in this guide retrieve data only. No deployment, schema change or business mutation is needed.